

Proyecto Backend

Integrantes

Rodrigo Riosei Galindo Merida

Josue Israel Quenallata Quiste

Isaias Daniel Aparicio Yañez

Descripcion

Este backend administra un marketplace de servicios basado en compra directa.

El cliente selecciona un servicio, elige un paquete publicado por un proveedor, toma un horario disponible y

genera una reserva. Luego se registra el pago, se confirma el cumplimiento del trabajo, se completa la

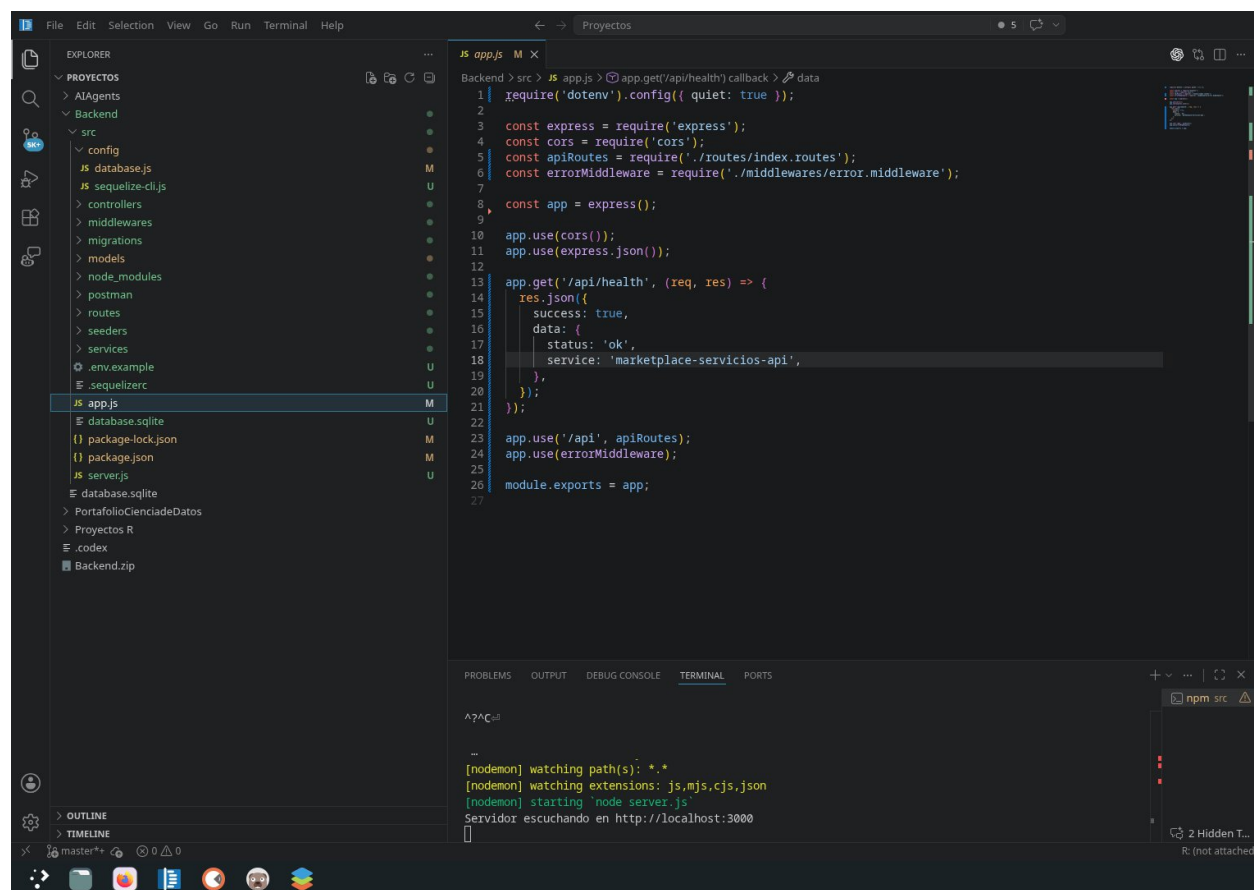
reserva y finalmente se habilita la reseña.

Entidades implementadas

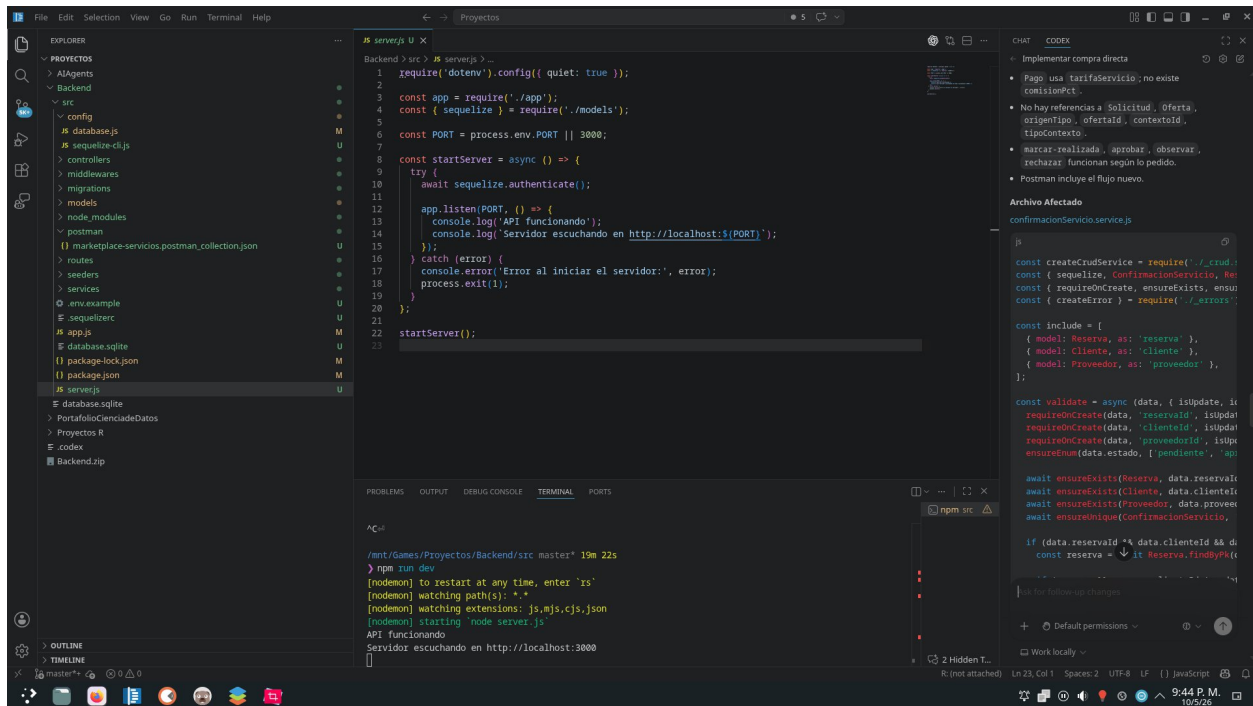
Entidad	¿Para qué sirve?
Usuario	Guarda los datos generales de una persona registrada en el sistema.
Cliente	Representa al usuario que reserva o compra servicios.
Proveedor	Representa al usuario que ofrece servicios dentro de la plataforma.
Categoría	Agrupar los servicios por tipo
Servicio	Define el tipo de servicio ofrecido, como limpieza profunda o mantenimiento.
Habilidad	Registra capacidades o requisitos asociados a proveedores y servicios.
PaqueteServicio	Representa una oferta concreta de un proveedor, con precio, duración y descripción.

DisponibilidadPI antilla	Define los horarios base en los que un proveedor suele estar disponible.
SlotDisponible	Representa un horario específico que puede ser reservado por un cliente.
Reserva	Registra la compra directa de un servicio en un horario determinado.
Pago	Guarda la información económica de la reserva, usando una tarifa de servicio fija.
ConfirmacionServicio	Permite verificar si el trabajo fue realizado y si el cliente quedó satisfecho.
Reseña	Guarda la calificación y comentario del cliente después de completar el servicio.
Mensaje	Permite la comunicación entre cliente y proveedor dentro de una reserva.

Estructura de carpetas

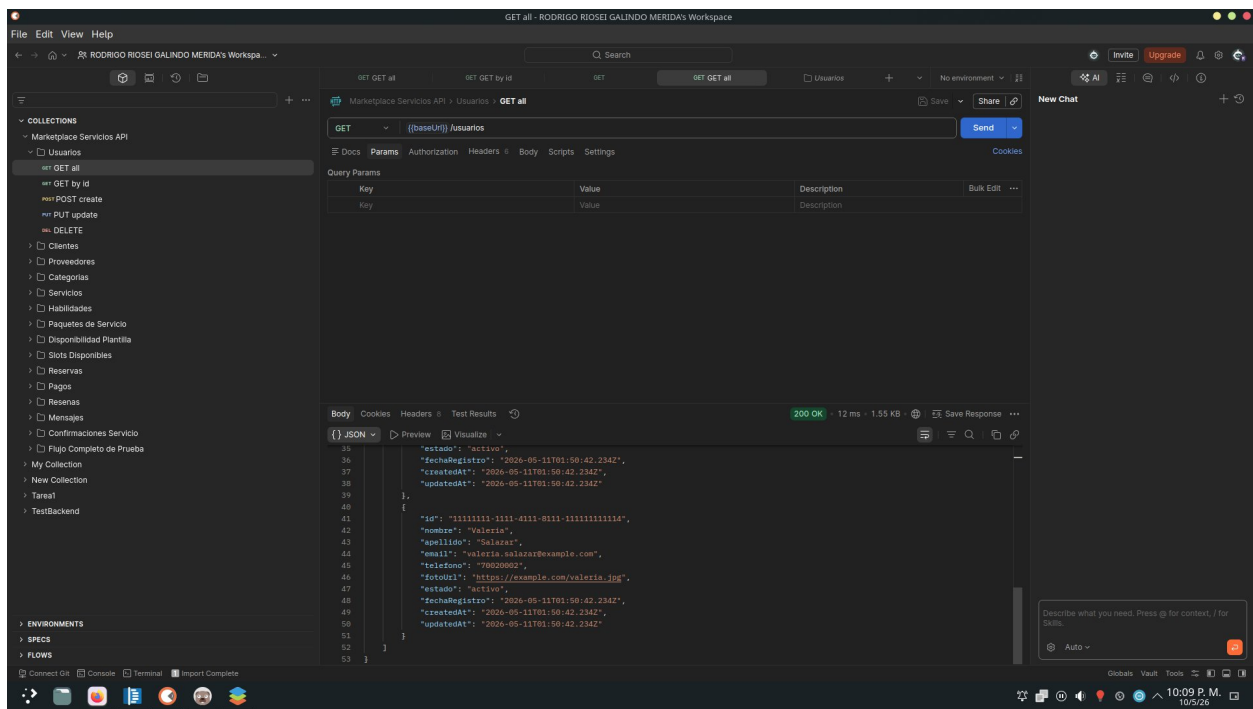


API funcionando

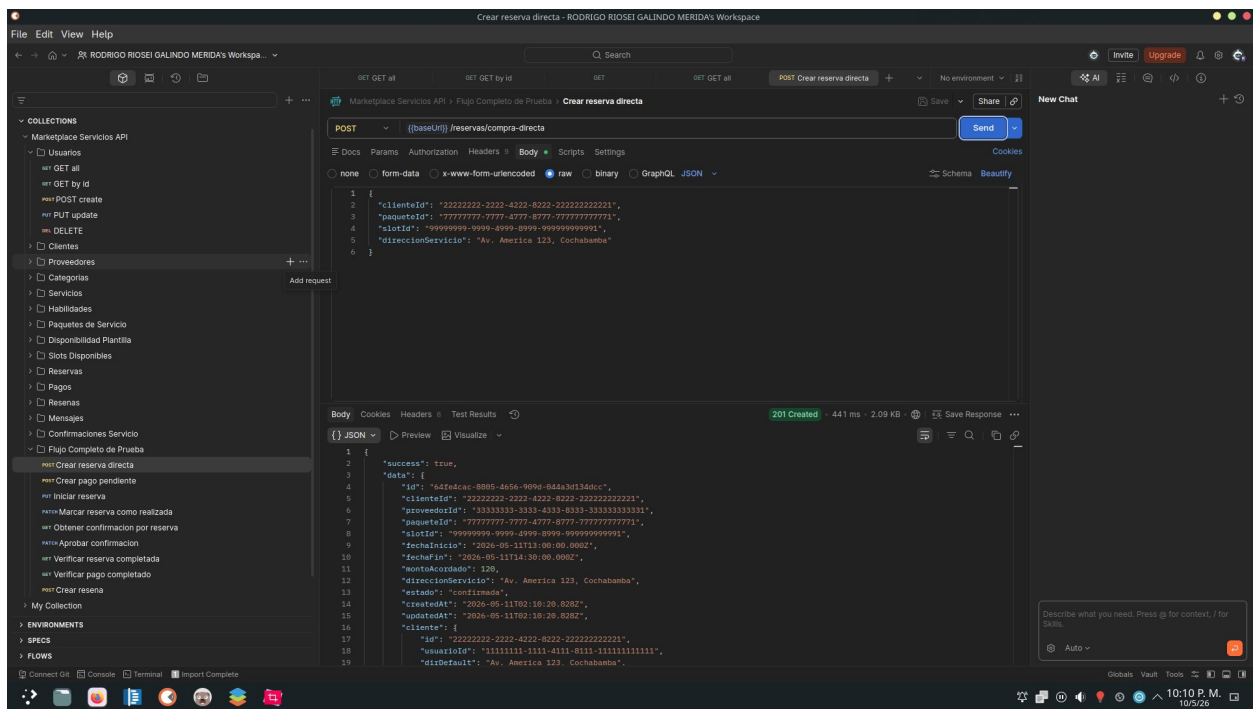


Pruebas POSTMAN

GET



Post Reserva



PUT

The screenshot shows a REST client interface with a sidebar on the left containing a collection of endpoints. The main panel displays a PUT request to the endpoint `PUT update` with a base URL of `[[baseUri]]/usuarios/[[usuarioId]]`. The request body is a JSON object:

```
1 {
2   "telefono": "70039999"
3 }
```

The response is a 200 OK status with a response time of 241 ms and a body of 605 B. The response body is a JSON object:

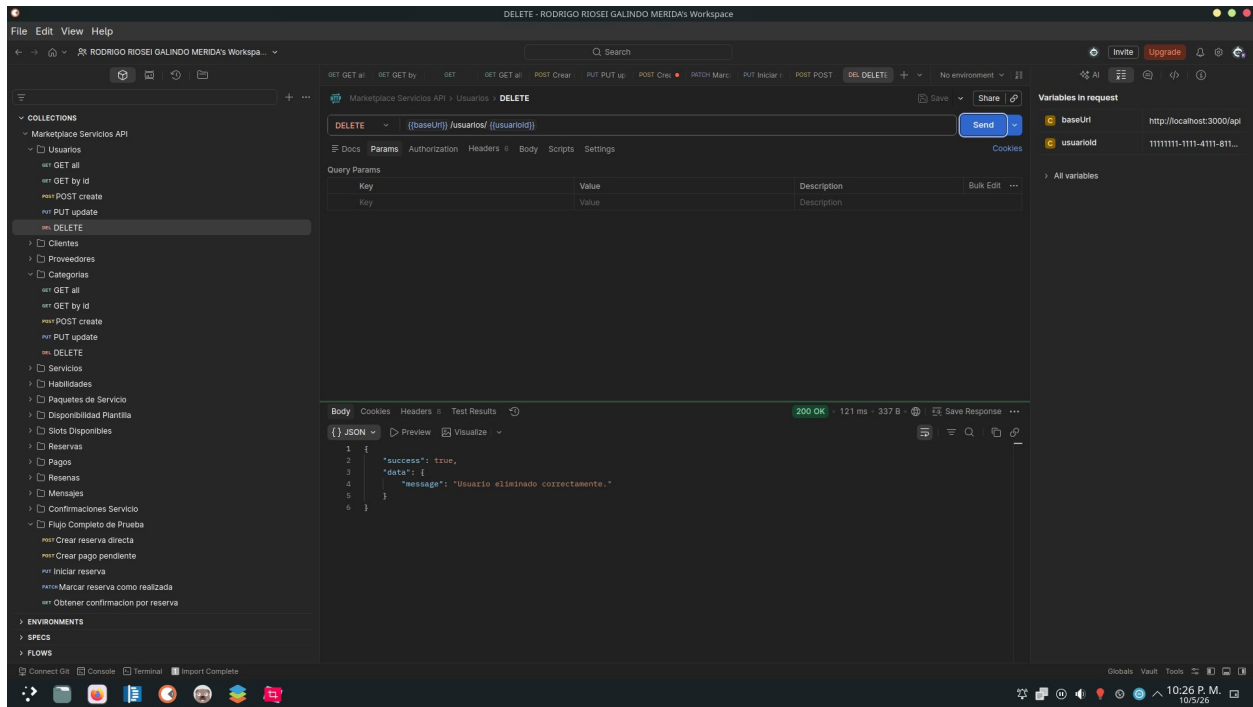
```
1 {
2   "success": true,
3   "data": {
4     "id": "11111111-1111-4111-8111-111111111111",
5     "nombre": "Ana",
6     "apellido": "Ruiz",
7     "email": "ana.ruiz@example.com",
8     "telefono": "70039999",
9     "fotoUrl": "https://example.com/ana.jpg",
10    "estado": "activo",
11    "fechaRegistro": "2026-05-11T01:50:42.234Z",
12    "createdAt": "2026-05-11T01:50:42.234Z",
13    "updatedAt": "2026-05-11T02:11:30.662Z"
14  }
15 }
```

PATCH

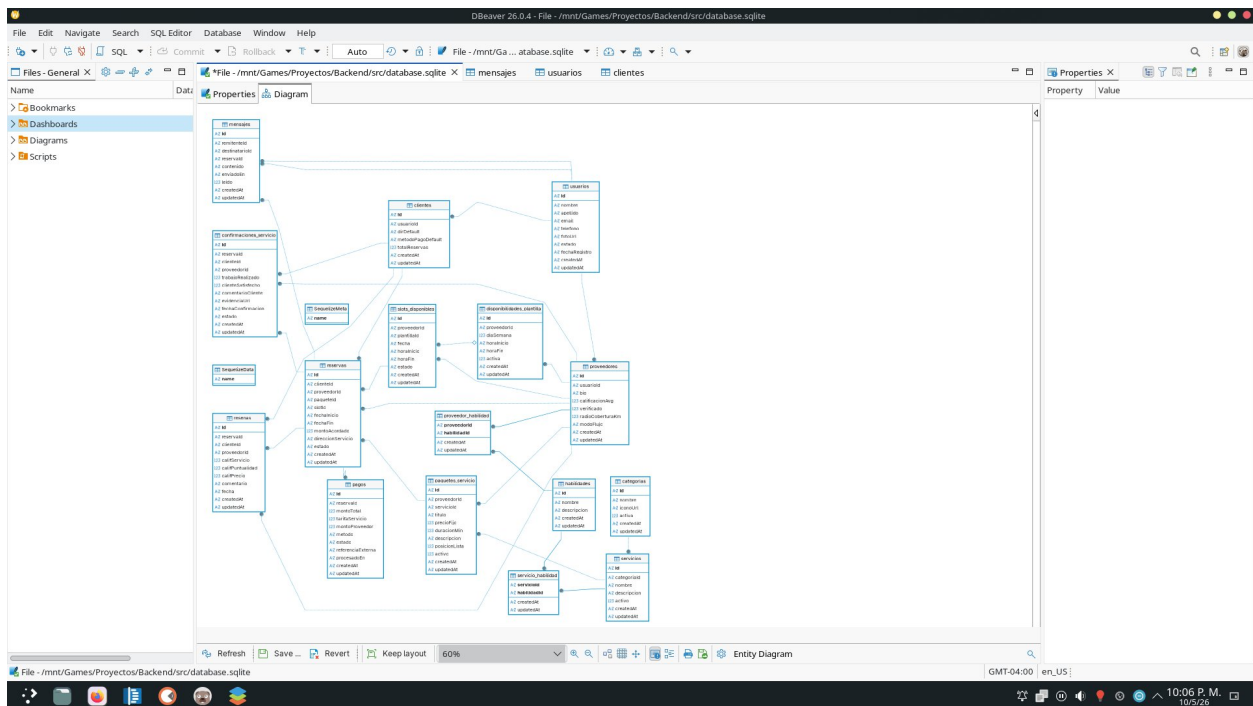
The screenshot shows a REST client interface with a sidebar on the left containing a collection of endpoints. The main panel displays a PATCH request to the endpoint `PATCH Marcar reserva como realizada` with a base URL of `[[baseUri]]/reservas/[[reservaId]]/marcar-realizada`. The request body is a JSON object:

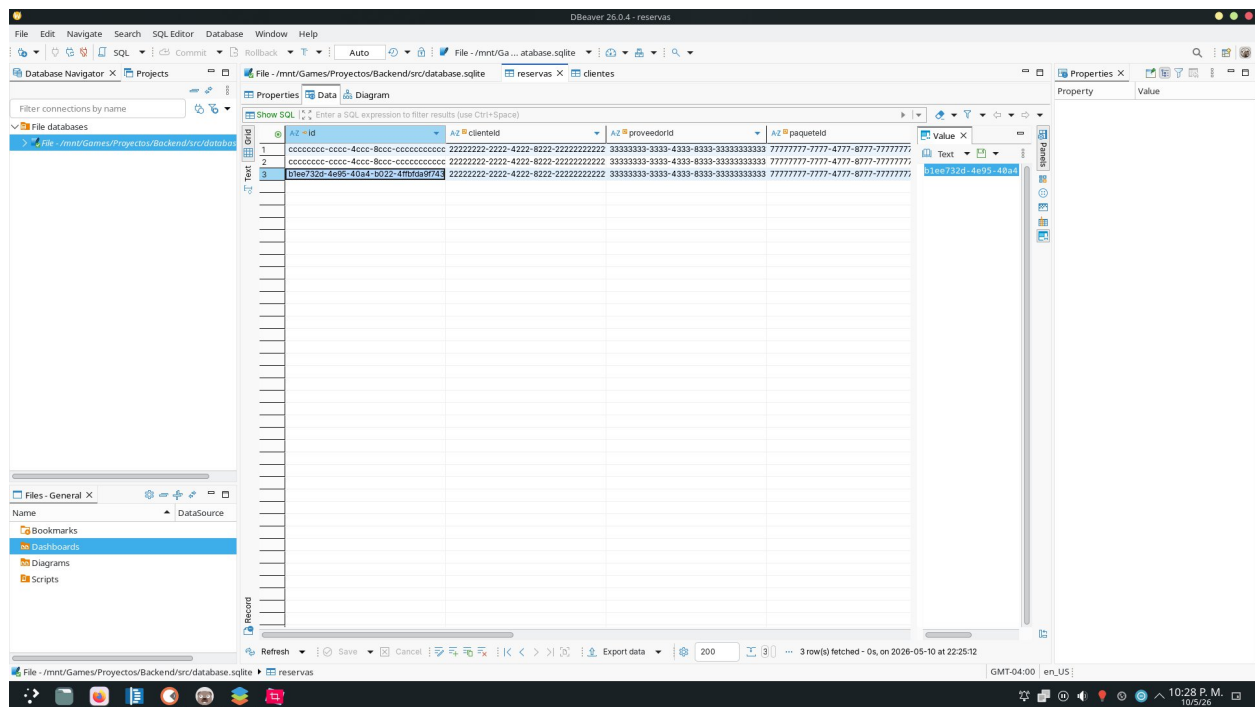
```
1 {
2   "success": true,
3   "data": {
4     "id": "11e732d-4e95-d8a4-9d22-4fffd99f743",
5     "clienteId": "22222222-2222-4222-8222-222222222221",
6     "proveedorId": "33333333-3333-4333-8333-333333333331",
7     "paqueteId": "77777777-7777-4777-8777-777777777771",
8     "estado": "pendiente confirmacion",
9     "fechaInicio": "2026-05-11T14:30:00.000Z",
10    "fechaFin": "2026-05-11T14:30:00.000Z",
11    "montoAcordado": 120,
12    "direccionServicio": "Av. America 123, Cochabamba",
13    "estado": "pendiente confirmacion",
14    "createdAt": "2026-05-11T02:26:05.801Z",
15    "updatedAt": "2026-05-11T02:26:07.217Z",
16    "cliente": {
17      "id": "22222222-2222-4222-8222-222222222221",
18      "usuarioId": "11111111-1111-4111-8111-111111111111",
19      "dirDefault": "Av. America 123, Cochabamba"
20    }
21  }
22 }
```

DELETE



Capturas DBeaver





Routes

Las routes son las rutas o caminos que usa la aplicación para recibir solicitudes del usuario. Básicamente conectan una URL con una función específica del sistema. Por ejemplo, cuando alguien entra a una página o presiona un botón, la ruta se encarga de dirigir esa acción al lugar correcto para que el sistema responda.

Controllers

Los controllers funcionan como el “intermediario” entre las rutas y la lógica del programa. Reciben la información que manda el usuario, la procesan y deciden qué respuesta devolver. En otras palabras, controlan lo que pasa cuando se ejecuta una acción dentro de la aplicación.

Services

Los services contienen la lógica principal del proyecto. Aquí es donde se realizan procesos importantes como cálculos, validaciones o consultas más complejas. Se usan para mantener el código más ordenado y evitar repetir funciones dentro de los controladores.

Models

Los models representan la estructura de los datos dentro del proyecto. Son como una plantilla que define cómo se guarda la información en la base de datos, por ejemplo usuarios, productos o pedidos. Gracias a ellos el sistema puede organizar y manejar los datos correctamente.

Database

La database o base de datos es donde se almacena toda la información del proyecto. Su función es guardar los datos de manera organizada para que el sistema pueda consultarlos, actualizarlos o eliminarlos cuando sea necesario. En el proyecto, la base de datos permite que la información no se pierda y que todo funcione de forma dinámica y automática.